

ICML 2023 | HD Maps Without the Rasterize-Then-Vectorize Detour

VectorMapNet, from Tsinghua, Shanghai Qi Zhi Institute, MIT, and Li Auto, is the first end-to-end vectorized HD map learner

Self-driving cars depend on high-definition (HD) semantic maps: the lane dividers, road boundaries, and pedestrian crossings that planning and prediction modules read as a prior. Today those maps are mostly annotated by hand offline, which is expensive and hard to scale across cities. A recent line of work takes a different route, letting a model label every pixel of a Bird's-Eye-View (BEV) grid and predict the map online, straight from the car's own sensors.

The catch is that a planner does not actually want a grid of pixels. A rasterized map has no notion of instances, so it cannot say which segments belong to the same lane; it is hard to keep spatially consistent; and, most importantly, it has to be run through a hand-designed post-processing step before it can be turned into the vectorized polylines that downstream modules consume. That rasterize-then-vectorize detour is exactly where accuracy and scalability leak away.

This ICML 2023 paper introduces VectorMapNet, which reframes map construction as a detection-then-generation problem: the model reads camera and LiDAR data and predicts a sparse set of vectorized polylines directly, skipping rasterization and post-processing entirely. It is the first end-to-end vectorized HD map learner.

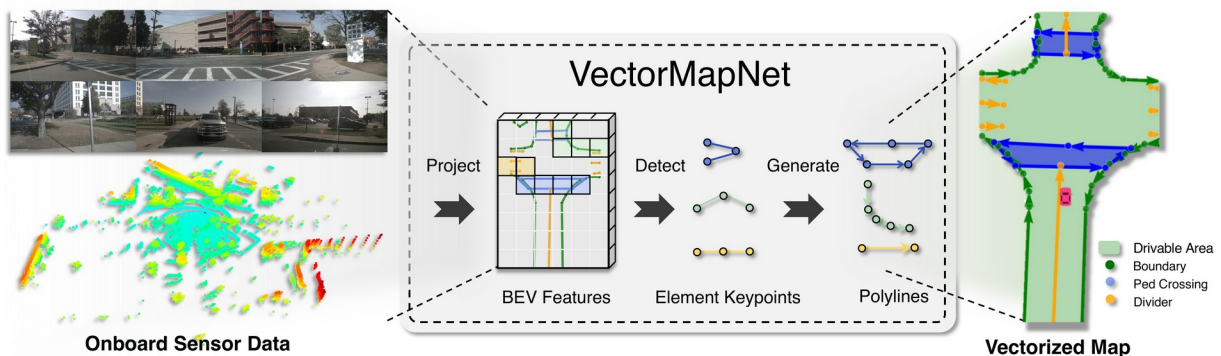


Figure 1. An overview of VectorMapNet. Onboard camera and LiDAR data (left) are encoded into a shared BEV feature space; learnable element queries detect where each map element sits; and a sparse set of polylines is generated from those detections to form the vectorized map (right). Because the polylines are directional, drivable and walkable areas can be inferred from them.

Paper: <https://arxiv.org/abs/2206.08920> | Code: https://github.com/Mrmoore98/VectorMapNet_code
lab.github.io/vectormapnet/ | Project page: <https://tsinghua-mars-lab.github.io/vectormapnet/>

Why skip the raster step

Start with why online map learning is appealing at all. Building a map on the fly from the vehicle's own cameras and LiDAR sidesteps both the annotation burden and the tens-of-centimeters localization error that offline HD maps incur at use time. But the authors push the argument one step further: a genuinely end-to-end system should output the representation downstream tasks use directly, rather than producing an intermediate artifact and then converting it.

The prior state of the art, HDMaNet, still predicts rasterized semantic, instance, and direction maps and then stitches those pixels into vectors with a hand-crafted algorithm, and that extra step caps both accuracy and scalability. VectorMapNet's motivation is to remove the detour outright by using the polyline as a single, unifying primitive. One polyline can represent points, lines, curves, and polygons, covering elements as varied as boundaries, dividers, and crossings, and the order of its vertices naturally encodes direction, which is exactly the signal a downstream planner needs to consume.

Three stages, from sensors to polylines

VectorMapNet is a chain of three stages trained together. The clearest way to read it is left to right across the architecture figure below.

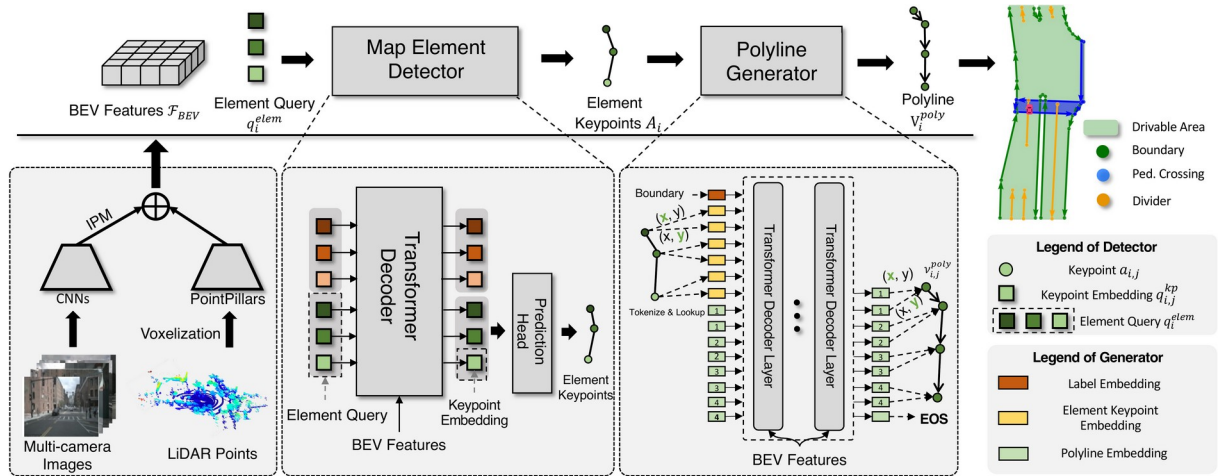


Figure 2. The network architecture of VectorMapNet. (a) BEV feature extractor: multi-camera images pass through a CNN and inverse perspective mapping (IPM) into the BEV space, LiDAR points pass through PointPillars, and the two are fused by concatenation. (b) Map element detector: a DETR-style Transformer decoder uses learnable element queries to locate and classify each element, emitting a few keypoints. (c) Polyline generator: an autoregressive Transformer decodes each element into ordered polyline vertices, one coordinate at a time, until it emits an EOS token.

The first stage is a BEV feature extractor that brings every modality into one top-down coordinate frame. The camera branch runs a ResNet and then an inverse perspective mapping module to project features onto the bird's-eye plane; the LiDAR branch runs PointPillars over the point cloud; the two feature maps are simply concatenated. The second stage is a map element detector adapted from DETR: it pairs a set of learnable element queries with

deformable attention to locate and classify every map element, whatever its size or shape, representing each by a small set of keypoints that the generator later turns into a full polyline.

The third stage is the polyline generator, and it is what turns detections into clean vectors. It is an autoregressive Transformer that takes each detected element and emits its polyline vertices one coordinate at a time. Vertex coordinates are quantized into discrete tokens and modeled with a categorical distribution, following PolyGen, so generating a polyline looks more like writing a sequence than regressing numbers. The whole model is trained end to end by summing a bipartite-matching detection loss and a negative-log-likelihood generation loss.

Two quiet design choices that matter

The paper uses ablations to isolate two design choices that are easy to overlook yet visibly change the result. The first is how to represent a map element with keypoints. The detector does not have to output a full polyline at once; it only sketches an element's rough position and extent with a few keypoints, leaving the rest to the generator. The authors compare three schemes, shown below.

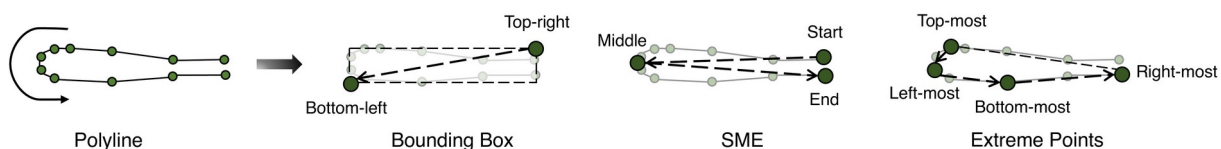


Figure 3. Three keypoint representations: Bounding Box ($k=2$, the bottom-left and top-right corners), SME ($k=3$, start-middle-end), and Extreme Points ($k=4$, the four extremal points). Here k is the number of keypoints per representation; the dashed arrows show their ordering. In practice the simplest two-point bounding box works best.

The result is a little counterintuitive: the simplest two-point Bounding Box ($k=2$) works best, beating the other two by 2.0 mAP under Fréchet distance and 7.3 mAP under Chamfer distance, so it becomes the default. The second choice is the training strategy. An autoregressive generator trained only with teacher forcing accumulates error at inference time through exposure bias. The authors instead use two-stage training: first teacher forcing, then fine-tuning on the model's own predicted keypoints. That fine-tuning adds +6.9 mAP for the camera-only model and +8.5 mAP for the fusion model.

Results: beating rasterized baselines across the board

Experiments run on two public datasets, nuScenes and Argoverse2, evaluating three element types: pedestrian crossings, lane dividers, and road boundaries. Following HMapNet's protocol, quality is average precision (AP) computed by matching predicted polylines against ground-truth ones; alongside the usual Chamfer distance, the paper introduces a Fréchet-distance AP that respects vertex order, at thresholds of 0.5, 1.0, and 1.5 m.

On nuScenes, VectorMapNet beats HMapNet under every sensor setting: +17.9 mAP camera-only, +9.9 mAP LiDAR-only, and +14.2 mAP with fusion. Fusion itself adds +4.3 mAP over camera-only and +11.2 mAP over LiDAR-only. With two-stage fine-tuning, the fusion model reaches 53.7 mAP, against 31.0 mAP for HMapNet in the same fusion setting. On Argoverse2

the margin is +14.6 mAP, and because that dataset carries height annotations, the model is also evaluated in 3D.

Table 1. Chamfer-distance AP (%) on nuScenes. VectorMapNet leads on all three element types, and fusion with fine-tuning reaches 53.7 mAP.

Method	Ped. crossing	Divider	Boundary	mAP
HMapNet (Camera)	14.4	21.7	33.0	23.0
HMapNet (LiDAR)	10.4	24.1	37.9	24.1
HMapNet (Fusion)	16.3	29.6	46.7	31.0
VectorMapNet (Camera)	36.1	47.3	39.3	40.9
VectorMapNet (LiDAR)	25.7	37.6	38.6	34.0
VectorMapNet (Fusion)	37.6	50.5	47.5	45.2
VectorMapNet (Fusion) + fine-tune	48.2	60.1	53.0	53.7

Beyond the numbers, the qualitative comparison makes the payoff of a vector representation easy to see. The figure below places ground truth, HMapNet, and VectorMapNet side by side on the same camera input.

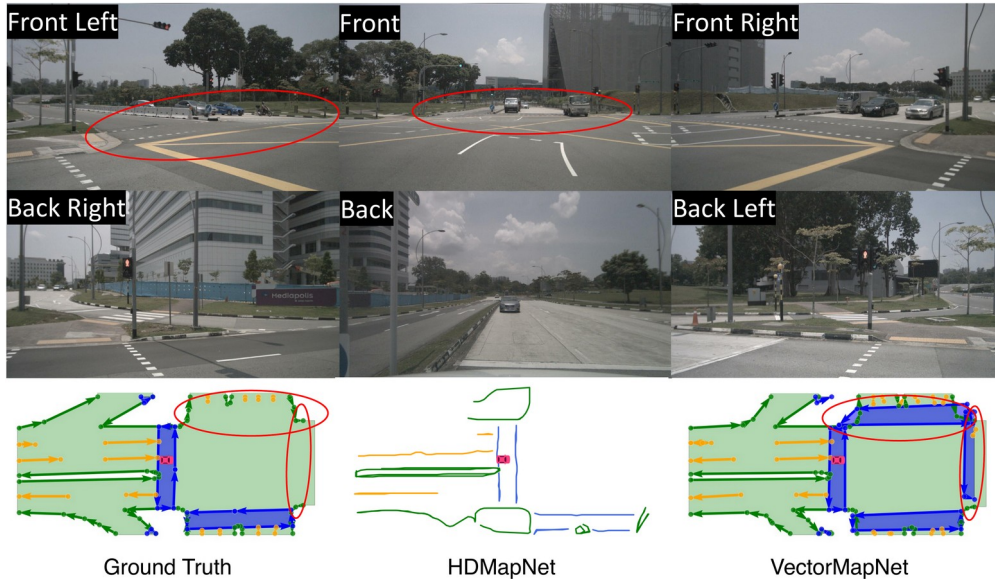


Figure 4. Qualitative comparison on nuScenes (all predictions from camera images). The red and blue ellipses highlight that VectorMapNet preserves sharp corners, and that the polyline representation avoids the ambiguous, self-looping results HMapNet produces; drivable area is inferred from the disjoint boundaries.

HMapNet's rasterized post-processing tends to produce breaks, self-intersections, and blurred corners; VectorMapNet's polylines come out cleaner, with crisper corners and geometry closer to the ground truth. The paper also notes an intriguing side effect: the model occasionally detects map elements that the ground-truth annotations missed, such as an unlabeled pedestrian crossing.

Useful downstream, with an honest boundary

A map is only worth predicting well if it actually helps downstream. The authors feed VectorMapNet's predicted maps into a motion-forecasting baseline: using past trajectories alone gives a minADE of 0.909, which drops to 0.826 with the predicted map, already close to the 0.779 obtained with human-annotated ground-truth maps, and within about 0.2% on miss rate. This suggests the end-to-end vector maps are already good enough to support a real downstream task.

The authors do not gloss over the limits. The model carries no temporal information, predicting each frame independently; when the cameras are occluded by nearby vehicles or the weather is bad, the camera-only variant misses or hallucinates elements and leans on LiDAR's depth cues to recover. The flexibility of the polyline primitive is a bonus: the same model even extends to predicting lane centerlines, with no structural change at all.

Takeaway

The lasting message of VectorMapNet is that HD map construction does not need a rasterize-then-vectorize detour. Framed as detection followed by autoregressive polyline generation, a single end-to-end network predicts clean, directional vector maps straight from onboard cameras and LiDAR, and outperforms the best rasterized pipelines by double-digit mAP. For teams working on online mapping, BEV perception, and autonomous-driving stacks, it offers a simpler route that lands much closer to what downstream modules actually need.